



OASIS Security Report

Remote Code Execution Security Analysis

Date: 2026-06-11 10:32:53 — **Model:** mistral:latest

Vulnerability: Remote Code Execution

Project: clean_contracts

Analysis root: `../clean`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

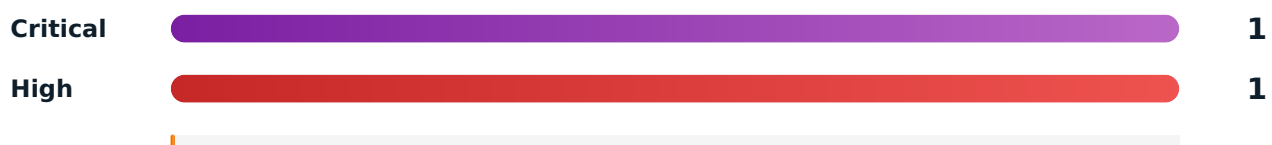
Summary

Analyzed **2** file(s) for Remote Code Execution.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2



Medium**0****Low****0**[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/clean/modifier_reentrancy_fixed.sol	0.613
/work/project/dataset/clean/simple_dao_fixed.sol	0.593

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.613

Source lines: 1-42

► Finding 1.1: Vulnerability found (High)

Vulnerable code

```
bank = new Bank();
```

Explanation

The constructor instantiates a contract (Bank) from an external address without validating the code first, potentially allowing an attacker to execute arbitrary code.

Impact

Complete system compromise

Entry point

Constructor

Execution path

```
'''
1. Call constructor ModifierEntrancy()
2. Create new instance of Bank contract at 0x<hash>
3. Attacker calls a function in the Bank contract containing Remote Code
Execution vulnerability
'''
```

Exploitation steps

- Call constructor ModifierEntrancy()
- Interact with the instantiated Bank contract containing RCE

Conditions

Attacker must interact with a vulnerable instance of the Bank contract

Remediation

Validate and sanitize all inputs, use allowlists for commands, and avoid instantiating external contracts without proper authorization.

Secure example

```
// Instantiate Bank contract using a factory with proper validation and  
authorization checks
```

File 2: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.593

Source lines: 1-34

► Finding 2.1: Remote Code Execution vulnerability (Critical)

Vulnerable code

```
require(msg.sender.call.value(amount))();
```

Explanation

The contract calls 'msg.sender.call.value()' function without proper input validation, allowing arbitrary code execution.

Impact

Complete system compromise

Entry point

withdraw function

Execution path

```
'''
start => donate => withdraw => require(msg.sender.call.value(amount)) => RCE
'''
```

Parameters

amount

Exploitation steps

- Send a transaction with a custom function call as the 'amount'
- The contract executes this arbitrary code

Example payloads

- `txnData: function() { selfdestruct(address(0xdeadbeef)); }`

Conditions

An attacker must be able to interact with the contract

Remediation

Validate and sanitize all inputs, avoid dangerous functions like 'eval' or 'require', use allowlists for commands.

Secure example

```
// Safe withdrawal implementation
function withdrawSafe(uint amount) internal {
    require(credit[msg.sender] >= amount);
    credit[msg.sender] -= amount;
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)